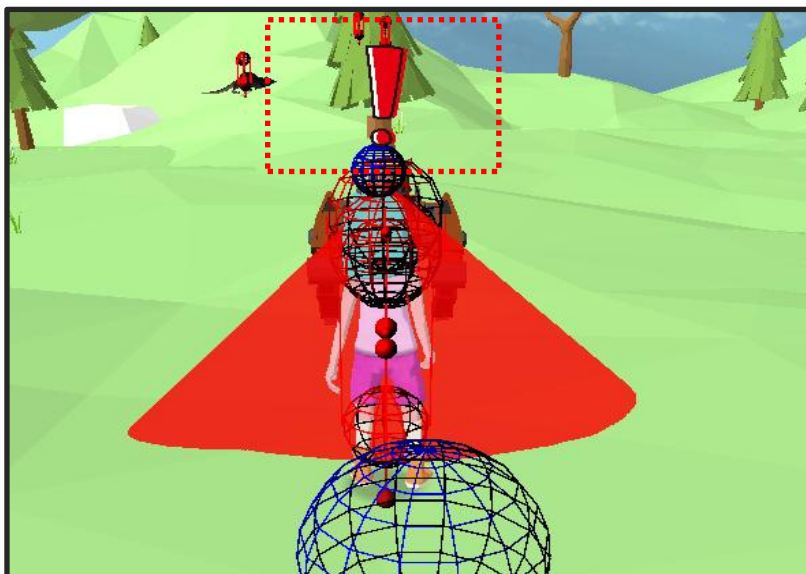


3D空間のUI (ビックリマーク)

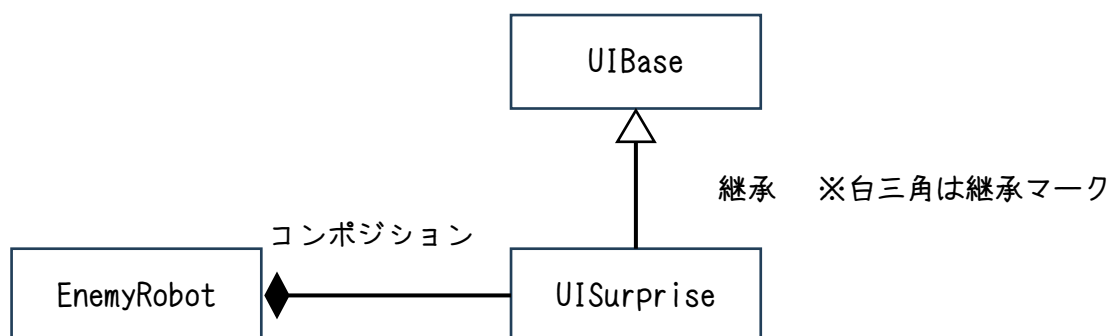
NPCが警戒状態に移行する処理を追加しましたが、
NPCの状態が急に遷移して、ユーザの理解が追い付かないケースがよくあります。

ゲームバランスにもよりますが、
学生時代のゲームでは、大げさでも構いませんので、
ゲーム内で何が起こっているのか、しっかりユーザに伝えた方が、
良いです。

ここでは、敵が警戒状態に移行した時によく使用される、
ビックリマークを3Dビルボードを使用し、
かつ、ただ描画するだけでなく、動き(イージング)を付けて、
より、ビックリしている感をプログラムで表現していきたいと思います。



クラス図



黒ひし形は、コンポジション。(Enemyが存在しないとUIも存在できない)
白ひし形は、集約。(仮に、Enemyが存在せずとも、UIが独立して存在できる場合)

```
UIBase.h
```

```
#pragma once
```

```
#include "../ActorBase.h"
```

```
class UIBase : public ActorBase  
{
```

```
public:
```

```
    // コンストラクタ
```

```
    UIBase(void);
```

```
    // デストラクタ
```

```
    virtual ~UIBase(void) override;
```

```
    // 有効・無効判定
```

```
    bool IsActive(void) const;
```

```
    // 有効・無効設定
```

```
    void SetActive(bool active);
```

```
    // 更新ステップ
```

```
    float GetStep(void) const;
```

```
protected:
```

```
    // 画像のハンドルID
```

```
    int img_;
```

```
    // 有効・無効
```

```
    bool active_;
```

```
    // 更新ステップ
```

```
    float step_;
```

```
};
```

UIBase.cpp

```
#include "UIBase.h"

UIBase::UIBase(void)
:
    ActorBase(),
    img_(-1),
    active_(false),
    step_(0.0f)

{
}

UIBase::~UIBase(void)
{
}

bool UIBase::IsActive(void) const
{
    return active_;
}

void UIBase::SetActive(bool active)
{
    active_ = active;
}

float UIBase::GetStep(void) const
{
    return step_;
}
```

UISurprise.h

```
#pragma once
#include "UIBase.h"
class Transform;

class UISurprise : public UIBase
{
}
```

public:

```
// コンストラクタ
UISurprise(
    Transform* follow, float localHeight,
    float bounceHeight, float imgSize);

// デストラクタ
~UISurprise(void) override;

// 更新
void Update(void) override;

// 描画
void Draw(void) override;

// 描画とイージング開始
void Start(void);
```

protected:

```
// リソースロード
void InitLoad(void) override;

// 大きさ、回転、座標の初期化
void InitTransform(void) override;

// 衝突判定の初期化
void InitCollider(void) override {}

// アニメーションの初期化
void InitAnimation(void) override {}

// 初期化後の個別処理
void InitPost(void) override;
```

private:

```
// 演出総時間
static constexpr float DURATION = 2.0f;
```

```

// バウンス演出時間
static constexpr float TIME_BOUNCE = 0.6f;

// バウンス強度
static constexpr float POW_BOUNCE = 9.0f;

// 追従対象
Transform* follow_;

// 画像の大きさ
float imgSize_;

// 追従対象からの相対位置(高さ)
float localHeight_;

// バウンス幅(高さ)
float bounceHeight_;

// イージング関数(バウンスアウト)
float EaseOutBounce(float t);

};

```

UISurprise.cpp

```

#include "../..../Manager/ResourceManager.h"
#include "../..../Manager/SceneManager.h"
#include "UISurprise.h"

```

```

UISurprise::UISurprise(
    Transform* follow, float localHeight,
    float bounceHeight, float imgSize)
:
    follow_(follow),
    localHeight_(localHeight),
    bounceHeight_(bounceHeight),
    imgSize_(imgSize)
{
}

```

```

UISurprise::~UISurprise(void)

```

```

{
}

void UISurprise::Update(void)
{
    if (!active_) return;

    // 演出進行
    step_ += scrMng_. GetDeltaTime();
    if (step_ > DURATION)
    {
        step_ = 1.0f;
        active_ = false;
    }

    // バウンスイージング
    float t = 1.0f;
    if (step_ < TIME_BOUNCE)
    {
        t = EaseOutBounce(step_ / TIME_BOUNCE);
    }

    // 高さ調整
    float easingOffset = t * bounceHeight_;

    // 追従
    transform_.pos = follow_->pos;

    // UIのベース座標にイージング移動量を加える
    transform_.pos.y += localHeight_ + easingOffset;
}

void UISurprise::Draw(void)
{
    if (!active_) return;

    // ビックリマーク描画
    DrawBillboard3D(

```

```

        transform_.pos,
        0.5f, 0.5f,
        imgSize_, 0.0f, img_, true);
    }

void UISurprise::Start(void)
{
    SetActive(true);
    step_ = 0.0f;
}

void UISurprise::InitLoad(void)
{
    // UI : ビックリマーク
    img_ = resMng_.Load(ResourceManager::SRC::UI_SURPRISE).handleId_;
}

void UISurprise::InitTransform(void)
{
}

void UISurprise::InitPost(void)
{
}

float UISurprise::EaseOutBounce(float t)
{
    // Robert Penner (ロバート・ペナー) 氏による
    // バウンス(跳ね返り)のイーasing

    float ret = 0.0f;

    // Step①～④の進行比率
    const float SPLIT = 2.75f;
    const float RATE1 = 1.0f / SPLIT;
    const float RATE2 = 2.0f / SPLIT;
    const float RATE3 = 2.5f / SPLIT;

```

```

if (t < RATE1)
{
    // Step① : 大ジャンプ
    ret = POW_BOUNCE * t * t;
}
else if (t < RATE2)
{
    // Step② : 跳ね返り
    t -= 1.2f / 2.75f;
    ret = POW_BOUNCE * t * t + 0.80f;
}
else if (t < RATE3)
{
    // Step③ : 小さな跳ね
    t -= 2.0f / 2.75f;
    ret = POW_BOUNCE * t * t + 0.95f;
}
else
{
    // Step④ : 微小バウンド
    t -= 2.3f / 2.75f;
    ret = POW_BOUNCE * t * t + 0.99f;
}

return ret;

}

```

解説

- DrawBillboard3D関数は、3D空間に2D画像を描画する関数
- イージングに使用しているバウンス式は、人がバウンスしているように見えるように工夫された有名な式を採用
※色々なアルゴリズムがあるので、興味がある人はぜひお試しを

【要件】

UISurpriseをEnemyRobotに組み込み、
ALERT状態の前に、SURPRISE状態を呼び出し、
ロボット上部にビックリマークを描画すること。

- ・ Surprise.pngをResourceManagerに設定すること
(Surprise.png は、TeamsのDataフォルダからダウンロード)
- ・ UI系のプログラムは、Actorフォルダ配下にUIフォルダを作成し、
その直下に作成すること

- ・ UISurpriseのインスタンスを生成する際、

高さは、120程度

バウンスによる上昇下降は、80程度

画像サイズは、96程度

- ・ 円錐視野内にプレイヤーが侵入した場合、
ALERT状態ではなく、SURPRISE状態へ遷移すること
- ・ SURPRISE状態遷移時

移動量ゼロ

IDLEアニメーション

移動方向は現在のまま

ビックリマーク機能を開始すること (Start関数)

- ・ SURPRISE状態から、1.5秒程度経過したら、ALERT状態へ遷移すること

【目標】

NPCの視野内にプレイヤーが侵入したら、
バウンスイージングを行う、ビックリマークUIが
NPCの頭上に描画され、その後、警戒状態に移行すること

